

```
-- =====
-- Dataset Name:
-- ITSM Service Desk Operations Analytics Dataset
--
-- CV Project Title:
-- ITSM Ticket, SLA & Support Operations Analytics using Python & PostgreSQL
--
-- Domain:
-- IT Service Management / Service Desk / Incident Management / SLA Analytics
--
-- Real-world Scenario:
-- A managed IT services company supports multiple enterprise clients.
-- The organization wants to analyze tickets, support agents, SLA tracking,
-- daily ITSM KPIs, management summaries, and monthly ticket activity logs.
--
-- Tables:
-- company                Company/client details
-- app_user               Support agent/user details
-- ticket                 Main ticket information
-- ticket_sla_log          SLA tracking records
-- daily_itsm_kpi          Daily operational KPIs
-- management_itsm_summary Management-level KPI summary
-- ticket_activity_log_2021_april April ticket activity logs
-- ticket_activity_log_2021_may   May ticket activity logs
-- ticket_activity_log_2021_june   June ticket activity logs
-- ticket_activity_log_2021_july   July ticket activity logs
-- =====
```

```
DROP SCHEMA IF EXISTS itsm_ops CASCADE;
CREATE SCHEMA itsm_ops;
SET search_path TO itsm_ops;
```

```
-- =====
-- 1. MASTER TABLES
-- =====
```

```
CREATE TABLE company (
    company_id SERIAL PRIMARY KEY,
    company_code VARCHAR(40) NOT NULL UNIQUE,
    company_name VARCHAR(150) NOT NULL,
    industry VARCHAR(80) NOT NULL,
    company_size VARCHAR(40) NOT NULL CHECK (company_size IN ('Small','Mid
Market','Enterprise','Strategic')),
    city VARCHAR(80) NOT NULL,
    state_name VARCHAR(80) NOT NULL,
    region VARCHAR(40) NOT NULL CHECK (region IN ('West','North','South','East','Central')),
    contract_type VARCHAR(60) NOT NULL CHECK (contract_type IN ('Standard Support','Premium
Support','Managed Services','Dedicated Desk')),
    support_plan VARCHAR(60) NOT NULL CHECK (support_plan IN
('Bronze','Silver','Gold','Platinum')),
    onboarding_date DATE NOT NULL,
    active_flag BOOLEAN NOT NULL DEFAULT TRUE
);
```

```
CREATE TABLE app_user (
    user_id SERIAL PRIMARY KEY,
    user_code VARCHAR(40) NOT NULL UNIQUE,
    full_name VARCHAR(120) NOT NULL,
    user_type VARCHAR(40) NOT NULL CHECK (user_type IN ('Support Agent','Team
Lead','Incident Manager','Problem Manager','Change Manager','End User')),
    support_team VARCHAR(80) NOT NULL CHECK (support_team IN ('Service Desk L1','Application
Support','Infrastructure Support','Network Support','Security Support','Database
Support','Field Support','Management')),
    skill_level VARCHAR(40) NOT NULL CHECK (skill_level IN
```

```
( 'Junior','Intermediate','Senior','Lead','Manager' )),
    shift_name VARCHAR(40) NOT NULL CHECK (shift_name IN
('Morning','Evening','Night','General')),
    city VARCHAR(80) NOT NULL,
    region VARCHAR(40) NOT NULL CHECK (region IN ('West','North','South','East','Central')),
    joining_date DATE NOT NULL,
    active_flag BOOLEAN NOT NULL DEFAULT TRUE
);
```

```
-- =====
-- 2. TRANSACTION TABLES
-- =====
```

```
CREATE TABLE ticket (
    ticket_id BIGSERIAL PRIMARY KEY,
    ticket_number VARCHAR(50) NOT NULL UNIQUE,
    company_id INT NOT NULL REFERENCES company(company_id),
    requester_user_id INT NOT NULL REFERENCES app_user(user_id),
    assigned_agent_id INT REFERENCES app_user(user_id),
    opened_at TIMESTAMP NOT NULL,
    resolved_at TIMESTAMP,
    closed_at TIMESTAMP,
    ticket_type VARCHAR(50) NOT NULL CHECK (ticket_type IN ('Incident','Service
Request','Problem','Change Request','Access Request')),
    category VARCHAR(80) NOT NULL CHECK (category IN
('Hardware','Software','Network','Email','Access','Database','Security','Cloud','ERP','Appli
cation')),
    sub_category VARCHAR(100) NOT NULL,
    priority VARCHAR(20) NOT NULL CHECK (priority IN ('P1','P2','P3','P4')),
    impact VARCHAR(30) NOT NULL CHECK (impact IN ('High','Medium','Low')),
    urgency VARCHAR(30) NOT NULL CHECK (urgency IN ('High','Medium','Low')),
    ticket_status VARCHAR(40) NOT NULL CHECK (ticket_status IN ('New','Assigned','In
Progress','Pending Customer','Resolved','Closed','Cancelled','Reopened')),
    source_channel VARCHAR(50) NOT NULL CHECK (source_channel IN
('Email','Phone','Portal','Chat','Monitoring Alert','Walk-in')),
    resolution_code VARCHAR(80),
    reopen_count INT NOT NULL DEFAULT 0,
    customer_satisfaction_rating INT CHECK (customer_satisfaction_rating BETWEEN 1 AND 5),
    created_month DATE NOT NULL
);
```

```
CREATE TABLE ticket_sla_log (
    sla_log_id BIGSERIAL PRIMARY KEY,
    ticket_id BIGINT NOT NULL REFERENCES ticket(ticket_id),
    sla_type VARCHAR(40) NOT NULL CHECK (sla_type IN ('First Response','Resolution')),
    sla_target_minutes INT NOT NULL,
    actual_minutes INT,
    sla_status VARCHAR(40) NOT NULL CHECK (sla_status IN ('Met','Breached','Running','Not
Applicable')),
    breach_minutes INT NOT NULL DEFAULT 0,
    measured_at TIMESTAMP NOT NULL
);
```

```
CREATE TABLE daily_itsm_kpi (
    daily_kpi_id BIGSERIAL PRIMARY KEY,
    kpi_date DATE NOT NULL,
    company_id INT NOT NULL REFERENCES company(company_id),
    support_team VARCHAR(80) NOT NULL,
    total_tickets INT NOT NULL,
    new_tickets INT NOT NULL,
    resolved_tickets INT NOT NULL,
    closed_tickets INT NOT NULL,
    reopened_tickets INT NOT NULL,
    backlog_tickets INT NOT NULL,
    sla_met_tickets INT NOT NULL,
```

```

        sla_breached_tickets INT NOT NULL,
        avg_resolution_minutes NUMERIC(10,2) NOT NULL,
        avg_first_response_minutes NUMERIC(10,2) NOT NULL,
        csat_avg NUMERIC(4,2)
    );

CREATE TABLE management_itsm_summary (
    summary_id BIGSERIAL PRIMARY KEY,
    summary_month DATE NOT NULL,
    company_id INT NOT NULL REFERENCES company(company_id),
    region VARCHAR(40) NOT NULL,
    support_plan VARCHAR(60) NOT NULL,
    total_tickets INT NOT NULL,
    incident_count INT NOT NULL,
    service_request_count INT NOT NULL,
    high_priority_count INT NOT NULL,
    resolved_count INT NOT NULL,
    closed_count INT NOT NULL,
    backlog_count INT NOT NULL,
    sla_met_count INT NOT NULL,
    sla_breached_count INT NOT NULL,
    sla_compliance_pct NUMERIC(6,2) NOT NULL,
    avg_resolution_hours NUMERIC(10,2) NOT NULL,
    avg_csat NUMERIC(4,2),
    escalation_count INT NOT NULL
);

-- Month-wise activity logs requested by user
CREATE TABLE ticket_activity_log_2021_april (
    activity_id BIGSERIAL PRIMARY KEY,
    ticket_id BIGINT NOT NULL REFERENCES ticket(ticket_id),
    activity_datetime TIMESTAMP NOT NULL,
    activity_type VARCHAR(60) NOT NULL,
    old_status VARCHAR(40),
    new_status VARCHAR(40),
    performed_by_user_id INT REFERENCES app_user(user_id),
    activity_notes TEXT,
    time_spent_minutes INT NOT NULL DEFAULT 0
);

CREATE TABLE ticket_activity_log_2021_may (LIKE ticket_activity_log_2021_april INCLUDING ALL);
CREATE TABLE ticket_activity_log_2021_june (LIKE ticket_activity_log_2021_april INCLUDING ALL);
CREATE TABLE ticket_activity_log_2021_july (LIKE ticket_activity_log_2021_april INCLUDING ALL);

-- =====
-- 3. MASTER DATA GENERATION
-- =====

INSERT INTO company (
    company_code, company_name, industry, company_size, city, state_name, region,
    contract_type, support_plan, onboarding_date, active_flag
)
SELECT
    'COMP-' || LPAD(gs::TEXT, 5, '0'),

    (ARRAY['Apex', 'Nexora', 'BluePeak', 'UrbanEdge', 'PrimeSoft', 'FinCore', 'MediPlus', 'RetailHub', 'CloudNova', 'GreenLeaf'])[1 + floor(random()*10)::INT] || ' Client ' || gs,
    (ARRAY['IT Services', 'Banking', 'Healthcare', 'Retail', 'Manufacturing', 'Education', 'Logistics', 'Telecom', 'Insurance', 'Real Estate'])[1 + floor(random()*10)::INT],
    (ARRAY['Small', 'Mid Market', 'Enterprise', 'Strategic'])[1 + floor(random()*4)::INT],
    loc.city,

```

```

loc.state_name,
loc.region,
(ARRAY['Standard Support','Premium Support','Managed Services','Dedicated Desk'])[1 +
floor(random()*4)::INT],
(ARRAY['Bronze','Silver','Gold','Platinum'])[1 + floor(random()*4)::INT],
DATE '2018-01-01' + floor(random()*1200)::INT,
CASE WHEN random() < 0.94 THEN TRUE ELSE FALSE END
FROM generate_series(1, 450) gs
CROSS JOIN LATERAL (
  SELECT * FROM (
    VALUES
      ('Pune','Maharashtra','West'),('Mumbai','Maharashtra','West'),
('Bengaluru','Karnataka','South'),
      ('Hyderabad','Telangana','South'),('Chennai','Tamil Nadu','South'),
('Delhi','Delhi','North'),
      ('Gurugram','Haryana','North'),('Ahmedabad','Gujarat','West'),('Kolkata','West
Bengal','East'),
      ('Noida','Uttar Pradesh','North'),('Jaipur','Rajasthan','North'),('Indore','Madhya
Pradesh','Central')
  ) AS x(city,state_name,region)
  ORDER BY random() + gs * 0
  LIMIT 1
) loc;

INSERT INTO app_user (
  user_code, full_name, user_type, support_team, skill_level, shift_name,
  city, region, joining_date, active_flag
)
SELECT
  'USR-' || LPAD(gs::TEXT, 6, '0'),

(ARRAY['Amit','Rahul','Priya','Sneha','Vikram','Anjali','Rohit','Pooja','Karan','Neha','Saga
r','Megha','Nikhil','Riya','Akshay','Komal'])[1 + floor(random()*16)::INT]
|| ' ' ||

(ARRAY['Sharma','Patil','Gupta','Joshi','Kadam','Kulkarni','Deshmukh','Mehta','Rao','Iyer','
Jadhav','Singh'])[1 + floor(random()*12)::INT],
CASE
  WHEN gs <= 850 THEN 'Support Agent'
  WHEN gs <= 980 THEN 'Team Lead'
  WHEN gs <= 1030 THEN 'Incident Manager'
  WHEN gs <= 1060 THEN 'Problem Manager'
  WHEN gs <= 1090 THEN 'Change Manager'
  ELSE 'End User'
END,
CASE
  WHEN gs > 1090 THEN 'Service Desk L1'
  ELSE (ARRAY['Service Desk L1','Application Support','Infrastructure
Support','Network Support','Security Support','Database Support','Field
Support','Management'])[1 + floor(random()*8)::INT]
END,
(ARRAY['Junior','Intermediate','Senior','Lead','Manager'])[1 + floor(random()*5)::INT],
(ARRAY['Morning','Evening','Night','General'])[1 + floor(random()*4)::INT],
loc.city,
loc.region,
DATE '2017-01-01' + floor(random()*1900)::INT,
CASE WHEN random() < 0.93 THEN TRUE ELSE FALSE END
FROM generate_series(1, 1250) gs
CROSS JOIN LATERAL (
  SELECT * FROM (
    VALUES
      ('Pune','West'),('Mumbai','West'),('Bengaluru','South'),('Hyderabad','South'),
('Chennai','South'),
      ('Delhi','North'),('Gurugram','North'),('Ahmedabad','West'),('Kolkata','East'),
('Noida','North'),('Indore','Central')

```

```

    ) AS x(city,region)
    ORDER BY random() + gs * 0
    LIMIT 1
) loc;

```

```

-- =====
-- 4. TICKET DATA GENERATION: APRIL TO JULY 2021
-- =====

```

```

INSERT INTO ticket (
    ticket_number, company_id, requester_user_id, assigned_agent_id, opened_at,
    resolved_at, closed_at, ticket_type, category, sub_category, priority,
    impact, urgency, ticket_status, source_channel, resolution_code,
    reopen_count, customer_satisfaction_rating, created_month
)
SELECT
    'INC-2021-' || LPAD(gs::TEXT, 8, '0'),
    comp.company_id,
    req.user_id,
    ag.user_id,
    opened_at,
    CASE WHEN ticket_status IN ('Resolved','Closed','Reopened') THEN opened_at +
(resolution_minutes || ' minutes')::INTERVAL ELSE NULL END,
    CASE WHEN ticket_status = 'Closed' THEN opened_at + ((resolution_minutes + 240 +
floor(random()*2880)::INT) || ' minutes')::INTERVAL ELSE NULL END,
    ticket_type,
    category,
    sub_category,
    priority,
    CASE WHEN priority IN ('P1','P2') THEN 'High' WHEN priority = 'P3' THEN 'Medium' ELSE
'Low' END,
    CASE WHEN priority = 'P1' THEN 'High' WHEN priority IN ('P2','P3') THEN 'Medium' ELSE
'Low' END,
    ticket_status,
    source_channel,
    CASE WHEN ticket_status IN ('Resolved','Closed','Reopened') THEN (ARRAY['Resolved by KB
Article','Password Reset Completed','Configuration Updated','Patch Applied','Access
Granted','Vendor Fix Applied','User Educated','Known Error Workaround'])[1 +
floor(random()*8)::INT] ELSE NULL END,
    CASE WHEN ticket_status = 'Reopened' THEN 1 + floor(random()*3)::INT ELSE 0 END,
    CASE WHEN ticket_status IN ('Resolved','Closed') AND random() < 0.62 THEN 1 +
floor(random()*5)::INT ELSE NULL END,
    DATE_TRUNC('month', opened_at)::DATE
FROM generate_series(1, 180000) gs
CROSS JOIN LATERAL (
    SELECT TIMESTAMP '2021-04-01 00:00:00'
        + (floor(random()*122)::INT || ' days')::INTERVAL
        + (floor(random()*24)::INT || ' hours')::INTERVAL
        + (floor(random()*60)::INT || ' minutes')::INTERVAL AS opened_at
) dt
JOIN LATERAL (
    SELECT company_id, support_plan
    FROM company
    WHERE active_flag = TRUE
    ORDER BY random() + gs * 0
    LIMIT 1
) comp ON TRUE
JOIN LATERAL (
    SELECT user_id
    FROM app_user
    WHERE user_type = 'End User' OR random() < 0.20
    ORDER BY random() + gs * 0
    LIMIT 1
) req ON TRUE
JOIN LATERAL (

```

```

SELECT user_id, support_team
FROM app_user
WHERE user_type IN ('Support Agent','Team Lead') AND active_flag = TRUE
ORDER BY random() + gs * 0
LIMIT 1
) ag ON TRUE
CROSS JOIN LATERAL (
    SELECT
        CASE
            WHEN random() < 0.58 THEN 'Incident'
            WHEN random() < 0.80 THEN 'Service Request'
            WHEN random() < 0.88 THEN 'Access Request'
            WHEN random() < 0.95 THEN 'Change Request'
            ELSE 'Problem'
        END AS ticket_type,

        (ARRAY['Hardware','Software','Network','Email','Access','Database','Security','Cloud','ERP',
        'Application'])[1 + floor(random()*10)::INT] AS category,
        CASE
            WHEN random() < 0.08 THEN 'P1'
            WHEN random() < 0.22 THEN 'P2'
            WHEN random() < 0.64 THEN 'P3'
            ELSE 'P4'
        END AS priority,
        CASE
            WHEN random() < 0.50 THEN 'Closed'
            WHEN random() < 0.72 THEN 'Resolved'
            WHEN random() < 0.82 THEN 'In Progress'
            WHEN random() < 0.90 THEN 'Pending Customer'
            WHEN random() < 0.95 THEN 'Assigned'
            WHEN random() < 0.98 THEN 'Reopened'
            ELSE 'Cancelled'
        END AS ticket_status,
        (ARRAY['Email','Phone','Portal','Chat','Monitoring Alert','Walk-in'])[1 +
        floor(random()*6)::INT] AS source_channel,
        60 + floor(random()*7200)::INT AS resolution_minutes
    ) x
CROSS JOIN LATERAL (
    SELECT CASE x.category
        WHEN 'Hardware' THEN (ARRAY['Laptop Issue','Printer Issue','Keyboard Mouse
Issue','Asset Replacement'])[1 + floor(random()*4)::INT]
        WHEN 'Software' THEN (ARRAY['Software Installation','Application Crash','License
Issue','Patch Update'])[1 + floor(random()*4)::INT]
        WHEN 'Network' THEN (ARRAY['VPN Issue','Wi-Fi Down','LAN Connectivity','Firewall
Request'])[1 + floor(random()*4)::INT]
        WHEN 'Email' THEN (ARRAY['Mailbox Access','Email Delivery Issue','Distribution
List','Spam Issue'])[1 + floor(random()*4)::INT]
        WHEN 'Access' THEN (ARRAY['Password Reset','Account Unlock','New Access','Access
Removal'])[1 + floor(random()*4)::INT]
        WHEN 'Database' THEN (ARRAY['DB Connectivity','Slow Query','Backup
Failure','Permission Issue'])[1 + floor(random()*4)::INT]
        WHEN 'Security' THEN (ARRAY['Malware Alert','Phishing Report','MFA Issue','Policy
Violation'])[1 + floor(random()*4)::INT]
        WHEN 'Cloud' THEN (ARRAY['VM Issue','Storage Alert','Cloud Access','Deployment
Failure'])[1 + floor(random()*4)::INT]
        WHEN 'ERP' THEN (ARRAY['SAP Login','Transaction Error','Report Issue','Authorization
Error'])[1 + floor(random()*4)::INT]
        ELSE (ARRAY['Bug Report','Feature Issue','API Error','Performance Issue'])[1 +
        floor(random()*4)::INT]
    END AS sub_category
    ) sub;

```

```

-- =====
-- 5. CONSISTENCY FIXES FOR TICKETS
-- =====

```

```

UPDATE ticket
SET resolved_at = NULL, closed_at = NULL, resolution_code = NULL,
customer_satisfaction_rating = NULL
WHERE ticket_status IN ('New','Assigned','In Progress','Pending Customer','Cancelled');

UPDATE ticket
SET closed_at = resolved_at + ((120 + floor(random()*2880)::INT) || ' minutes')::INTERVAL
WHERE ticket_status = 'Closed' AND resolved_at IS NOT NULL AND (closed_at IS NULL OR
closed_at < resolved_at);

UPDATE ticket
SET customer_satisfaction_rating = 1 + floor(random()*5)::INT
WHERE ticket_status IN ('Resolved','Closed') AND customer_satisfaction_rating IS NULL AND
random() < 0.55;

-- =====
-- 6. SLA LOG GENERATION
-- =====

INSERT INTO ticket_sla_log (
    ticket_id, sla_type, sla_target_minutes, actual_minutes, sla_status, breach_minutes,
    measured_at
)
SELECT
    t.ticket_id,
    sla_type,
    target_minutes,
    actual_minutes,
    CASE
        WHEN actual_minutes IS NULL THEN 'Running'
        WHEN actual_minutes <= target_minutes THEN 'Met'
        ELSE 'Breach'
    END AS sla_status,
    GREATEST(COALESCE(actual_minutes,0) - target_minutes, 0) AS breach_minutes,
    t.opened_at + (COALESCE(actual_minutes, target_minutes) || ' minutes')::INTERVAL
FROM ticket t
CROSS JOIN LATERAL (
    VALUES ('First Response'), ('Resolution')
) s(sla_type)
CROSS JOIN LATERAL (
    SELECT CASE
        WHEN s.sla_type = 'First Response' THEN
            CASE t.priority WHEN 'P1' THEN 15 WHEN 'P2' THEN 30 WHEN 'P3' THEN 120 ELSE 240
        ELSE
            CASE t.priority WHEN 'P1' THEN 240 WHEN 'P2' THEN 480 WHEN 'P3' THEN 1440 ELSE
2880 END
        END AS target_minutes
    ) trg
CROSS JOIN LATERAL (
    SELECT CASE
        WHEN s.sla_type = 'First Response' THEN 5 + floor(random()*420)::INT
        WHEN t.ticket_status IN ('Resolved','Closed','Reopened') AND t.resolved_at IS NOT
NULL THEN EXTRACT(EPOCH FROM (t.resolved_at - t.opened_at))/60
        ELSE NULL
    END)::INT AS actual_minutes
) act;

-- =====
-- 7. ACTIVITY LOGS BY MONTH
-- =====

INSERT INTO ticket_activity_log_2021_april (
    ticket_id, activity_datetime, activity_type, old_status, new_status,

```

```

        performed_by_user_id, activity_notes, time_spent_minutes
    )
SELECT t.ticket_id, t.opened_at, 'Ticket Created', NULL, 'New', t.requester_user_id,
       'Ticket created by requester.', 0
FROM ticket t
WHERE t.opened_at >= TIMESTAMP '2021-04-01' AND t.opened_at < TIMESTAMP '2021-05-01';

INSERT INTO ticket_activity_log_2021_april (ticket_id, activity_datetime, activity_type,
old_status, new_status, performed_by_user_id, activity_notes, time_spent_minutes)
SELECT t.ticket_id,
       t.opened_at + ((15 + floor(random()*480)::INT) || ' minutes')::INTERVAL,
       'Status Updated', 'New', 'In Progress', t.assigned_agent_id,
       'Ticket triaged and assigned to support queue.', 5 + floor(random()*45)::INT
FROM ticket t
WHERE t.opened_at >= TIMESTAMP '2021-04-01' AND t.opened_at < TIMESTAMP '2021-05-01' AND
t.assigned_agent_id IS NOT NULL;

INSERT INTO ticket_activity_log_2021_april (ticket_id, activity_datetime, activity_type,
old_status, new_status, performed_by_user_id, activity_notes, time_spent_minutes)
SELECT t.ticket_id, t.resolved_at, 'Resolution Added', 'In
Progress', 'Resolved', t.assigned_agent_id,
       COALESCE(t.resolution_code, 'Resolution details updated.'), 15 +
       floor(random()*180)::INT
FROM ticket t
WHERE t.opened_at >= TIMESTAMP '2021-04-01' AND t.opened_at < TIMESTAMP '2021-05-01' AND
t.resolved_at IS NOT NULL;

INSERT INTO ticket_activity_log_2021_may (ticket_id, activity_datetime, activity_type,
old_status, new_status, performed_by_user_id, activity_notes, time_spent_minutes)
SELECT t.ticket_id, t.opened_at, 'Ticket Created', NULL, 'New', t.requester_user_id, 'Ticket
created by requester.', 0
FROM ticket t
WHERE t.opened_at >= TIMESTAMP '2021-05-01' AND t.opened_at < TIMESTAMP '2021-06-01';

INSERT INTO ticket_activity_log_2021_may (ticket_id, activity_datetime, activity_type,
old_status, new_status, performed_by_user_id, activity_notes, time_spent_minutes)
SELECT t.ticket_id, t.opened_at + ((15 + floor(random()*480)::INT) || ' minutes')::INTERVAL,
       'Status Updated', 'New', 'In Progress', t.assigned_agent_id, 'Ticket triaged and assigned to
support queue.', 5 + floor(random()*45)::INT
FROM ticket t
WHERE t.opened_at >= TIMESTAMP '2021-05-01' AND t.opened_at < TIMESTAMP '2021-06-01' AND
t.assigned_agent_id IS NOT NULL;

INSERT INTO ticket_activity_log_2021_may (ticket_id, activity_datetime, activity_type,
old_status, new_status, performed_by_user_id, activity_notes, time_spent_minutes)
SELECT t.ticket_id, t.resolved_at, 'Resolution Added', 'In
Progress', 'Resolved', t.assigned_agent_id, COALESCE(t.resolution_code, 'Resolution details
updated.'), 15 + floor(random()*180)::INT
FROM ticket t
WHERE t.opened_at >= TIMESTAMP '2021-05-01' AND t.opened_at < TIMESTAMP '2021-06-01' AND
t.resolved_at IS NOT NULL;

INSERT INTO ticket_activity_log_2021_june (ticket_id, activity_datetime, activity_type,
old_status, new_status, performed_by_user_id, activity_notes, time_spent_minutes)
SELECT t.ticket_id, t.opened_at, 'Ticket Created', NULL, 'New', t.requester_user_id, 'Ticket
created by requester.', 0
FROM ticket t
WHERE t.opened_at >= TIMESTAMP '2021-06-01' AND t.opened_at < TIMESTAMP '2021-07-01';

INSERT INTO ticket_activity_log_2021_june (ticket_id, activity_datetime, activity_type,
old_status, new_status, performed_by_user_id, activity_notes, time_spent_minutes)
SELECT t.ticket_id, t.opened_at + ((15 + floor(random()*480)::INT) || ' minutes')::INTERVAL,
       'Status Updated', 'New', 'In Progress', t.assigned_agent_id, 'Ticket triaged and assigned to
support queue.', 5 + floor(random()*45)::INT
FROM ticket t

```

```
WHERE t.opened_at >= TIMESTAMP '2021-06-01' AND t.opened_at < TIMESTAMP '2021-07-01' AND  
t.assigned_agent_id IS NOT NULL;
```

```
INSERT INTO ticket_activity_log_2021_june (ticket_id, activity_datetime, activity_type,  
old_status, new_status, performed_by_user_id, activity_notes, time_spent_minutes)  
SELECT t.ticket_id, t.resolved_at, 'Resolution Added', 'In  
Progress', 'Resolved', t.assigned_agent_id, COALESCE(t.resolution_code, 'Resolution details  
updated.'), 15 + floor(random()*180)::INT  
FROM ticket t  
WHERE t.opened_at >= TIMESTAMP '2021-06-01' AND t.opened_at < TIMESTAMP '2021-07-01' AND  
t.resolved_at IS NOT NULL;
```

```
INSERT INTO ticket_activity_log_2021_july (ticket_id, activity_datetime, activity_type,  
old_status, new_status, performed_by_user_id, activity_notes, time_spent_minutes)  
SELECT t.ticket_id, t.opened_at, 'Ticket Created', NULL, 'New', t.requester_user_id, 'Ticket  
created by requester.', 0  
FROM ticket t  
WHERE t.opened_at >= TIMESTAMP '2021-07-01' AND t.opened_at < TIMESTAMP '2021-08-01';
```

```
INSERT INTO ticket_activity_log_2021_july (ticket_id, activity_datetime, activity_type,  
old_status, new_status, performed_by_user_id, activity_notes, time_spent_minutes)  
SELECT t.ticket_id, t.opened_at + ((15 + floor(random()*480)::INT) || ' minutes')::INTERVAL,  
'Status Updated', 'New', 'In Progress', t.assigned_agent_id, 'Ticket triaged and assigned to  
support queue.', 5 + floor(random()*45)::INT  
FROM ticket t  
WHERE t.opened_at >= TIMESTAMP '2021-07-01' AND t.opened_at < TIMESTAMP '2021-08-01' AND  
t.assigned_agent_id IS NOT NULL;
```

```
INSERT INTO ticket_activity_log_2021_july (ticket_id, activity_datetime, activity_type,  
old_status, new_status, performed_by_user_id, activity_notes, time_spent_minutes)  
SELECT t.ticket_id, t.resolved_at, 'Resolution Added', 'In  
Progress', 'Resolved', t.assigned_agent_id, COALESCE(t.resolution_code, 'Resolution details  
updated.'), 15 + floor(random()*180)::INT  
FROM ticket t  
WHERE t.opened_at >= TIMESTAMP '2021-07-01' AND t.opened_at < TIMESTAMP '2021-08-01' AND  
t.resolved_at IS NOT NULL;
```

```
-- =====  
-- 8. DAILY KPI GENERATED FROM ACTUAL TICKET + SLA FACTS  
-- =====
```

```
INSERT INTO daily_itsm_kpi (  
    kpi_date, company_id, support_team, total_tickets, new_tickets,  
    resolved_tickets, closed_tickets, reopened_tickets, backlog_tickets,  
    sla_met_tickets, sla_breached_tickets, avg_resolution_minutes,  
    avg_first_response_minutes, csat_avg  
)  
SELECT  
    d.kpi_date,  
    c.company_id,  
    st.support_team,  
    COUNT(t.ticket_id) AS total_tickets,  
    COUNT(t.ticket_id) FILTER (WHERE t.ticket_status = 'New') AS new_tickets,  
    COUNT(t.ticket_id) FILTER (WHERE t.ticket_status = 'Resolved') AS resolved_tickets,  
    COUNT(t.ticket_id) FILTER (WHERE t.ticket_status = 'Closed') AS closed_tickets,  
    COUNT(t.ticket_id) FILTER (WHERE t.ticket_status = 'Reopened') AS reopened_tickets,  
    COUNT(t.ticket_id) FILTER (WHERE t.ticket_status IN ('New', 'Assigned', 'In  
Progress', 'Pending Customer', 'Reopened')) AS backlog_tickets,  
    COUNT(DISTINCT t.ticket_id) FILTER (WHERE sl.sla_status = 'Met') AS sla_met_tickets,  
    COUNT(DISTINCT t.ticket_id) FILTER (WHERE sl.sla_status = 'Breached') AS  
sla_breached_tickets,  
    COALESCE(ROUND(AVG(EXTRACT(EPOCH FROM (t.resolved_at - t.opened_at))/60) FILTER (WHERE  
t.resolved_at IS NOT NULL), 2), 0) AS avg_resolution_minutes,  
    COALESCE(ROUND(AVG(sl.actual_minutes) FILTER (WHERE sl.sla_type = 'First Response' AND  
sl.actual_minutes IS NOT NULL), 2), 0) AS avg_first_response_minutes,
```

```

ROUND(AVG(t.customer_satisfaction_rating) FILTER (WHERE t.customer_satisfaction_rating
IS NOT NULL),2) AS csat_avg
FROM (
    SELECT generate_series(DATE '2021-04-01', DATE '2021-07-31', INTERVAL '1 day')::DATE AS
kpi_date
) d
CROSS JOIN company c
CROSS JOIN (
    SELECT DISTINCT support_team FROM app_user WHERE support_team <> 'Management'
) st
LEFT JOIN ticket t
    ON t.company_id = c.company_id
    AND DATE(t.opened_at) = d.kpi_date
LEFT JOIN app_user au
    ON t.assigned_agent_id = au.user_id
    AND au.support_team = st.support_team
LEFT JOIN ticket_sla_log sl
    ON t.ticket_id = sl.ticket_id
GROUP BY d.kpi_date, c.company_id, st.support_team;

-- Remove zero-noise KPI rows to keep dashboard practical
DELETE FROM daily_itsm_kpi
WHERE total_tickets = 0;

-- =====
-- 9. MANAGEMENT SUMMARY FROM ACTUAL FACTS
-- =====

INSERT INTO management_itsm_summary (
    summary_month, company_id, region, support_plan, total_tickets,
    incident_count, service_request_count, high_priority_count, resolved_count,
    closed_count, backlog_count, sla_met_count, sla_breached_count,
    sla_compliance_pct, avg_resolution_hours, avg_csat, escalation_count
)
SELECT
    DATE_TRUNC('month', t.opened_at)::DATE AS summary_month,
    c.company_id,
    c.region,
    c.support_plan,
    COUNT(*) AS total_tickets,
    COUNT(*) FILTER (WHERE t.ticket_type = 'Incident') AS incident_count,
    COUNT(*) FILTER (WHERE t.ticket_type = 'Service Request') AS service_request_count,
    COUNT(*) FILTER (WHERE t.priority IN ('P1','P2')) AS high_priority_count,
    COUNT(*) FILTER (WHERE t.ticket_status = 'Resolved') AS resolved_count,
    COUNT(*) FILTER (WHERE t.ticket_status = 'Closed') AS closed_count,
    COUNT(*) FILTER (WHERE t.ticket_status IN ('New','Assigned','In Progress','Pending
Customer','Reopened')) AS backlog_count,
    COUNT(DISTINCT t.ticket_id) FILTER (WHERE sl.sla_status = 'Met') AS sla_met_count,
    COUNT(DISTINCT t.ticket_id) FILTER (WHERE sl.sla_status = 'Breached') AS
sla_breached_count,
    ROUND(
        100.0 * COUNT(DISTINCT t.ticket_id) FILTER (WHERE sl.sla_status = 'Met')
        / NULLIF(COUNT(DISTINCT t.ticket_id) FILTER (WHERE sl.sla_status IN
('Met','Breached')),0), 2
    ) AS sla_compliance_pct,
    COALESCE(ROUND(AVG(EXTRACT(EPOCH FROM (t.resolved_at - t.opened_at)))/3600) FILTER (WHERE
t.resolved_at IS NOT NULL),2),0) AS avg_resolution_hours,
    ROUND(AVG(t.customer_satisfaction_rating) FILTER (WHERE t.customer_satisfaction_rating
IS NOT NULL),2) AS avg_csat,
    COUNT(*) FILTER (WHERE t.priority = 'P1' OR t.reopen_count > 0) AS escalation_count
FROM ticket t
JOIN company c ON t.company_id = c.company_id
LEFT JOIN ticket_sla_log sl ON t.ticket_id = sl.ticket_id AND sl.sla_type = 'Resolution'
GROUP BY DATE_TRUNC('month', t.opened_at)::DATE, c.company_id, c.region, c.support_plan;

```

```
-- =====
-- 10. INDEXES
-- =====
```

```
CREATE INDEX idx_ticket_company_date ON ticket(company_id, opened_at);
CREATE INDEX idx_ticket_status_priority ON ticket(ticket_status, priority);
CREATE INDEX idx_ticket_agent ON ticket(assigned_agent_id);
CREATE INDEX idx_sla_ticket_type ON ticket_sla_log(ticket_id, sla_type);
CREATE INDEX idx_daily_kpi_date_company ON daily_itsm_kpi(kpi_date, company_id);
CREATE INDEX idx_management_summary_month ON management_itsm_summary(summary_month,
company_id);
CREATE INDEX idx_april_activity_ticket ON ticket_activity_log_2021_april(ticket_id,
activity_datetime);
CREATE INDEX idx_may_activity_ticket ON ticket_activity_log_2021_may(ticket_id,
activity_datetime);
CREATE INDEX idx_june_activity_ticket ON ticket_activity_log_2021_june(ticket_id,
activity_datetime);
CREATE INDEX idx_july_activity_ticket ON ticket_activity_log_2021_july(ticket_id,
activity_datetime);
```

```
-- =====
-- 11. POWER BI READY VIEWS
-- =====
```

```
CREATE OR REPLACE VIEW vw_ticket_detail AS
SELECT
    t.ticket_id,
    t.ticket_number,
    c.company_name,
    c.industry,
    c.company_size,
    c.region AS company_region,
    c.support_plan,
    au.full_name AS assigned_agent,
    au.support_team,
    au.skill_level,
    t.opened_at,
    DATE_TRUNC('month', t.opened_at)::DATE AS ticket_month,
    t.resolved_at,
    t.closed_at,
    t.ticket_type,
    t.category,
    t.sub_category,
    t.priority,
    t.impact,
    t.urgency,
    t.ticket_status,
    t.source_channel,
    t.reopen_count,
    t.customer_satisfaction_rating,
    ROUND(EXTRACT(EPOCH FROM (t.resolved_at - t.opened_at))/3600,2) AS resolution_hours
FROM ticket t
JOIN company c ON t.company_id = c.company_id
LEFT JOIN app_user au ON t.assigned_agent_id = au.user_id;

CREATE OR REPLACE VIEW vw_sla_performance AS
SELECT
    DATE_TRUNC('month', t.opened_at)::DATE AS ticket_month,
    c.company_name,
    c.support_plan,
    au.support_team,
    t.priority,
    sl.sla_type,
    sl.sla_status,
    COUNT(*) AS sla_records,
```

```

COUNT(*) FILTER (WHERE sl.sla_status = 'Met') AS met_count,
COUNT(*) FILTER (WHERE sl.sla_status = 'Breached') AS breached_count,
ROUND(AVG(sl.actual_minutes) FILTER (WHERE sl.actual_minutes IS NOT NULL),2) AS
avg_actual_minutes,
ROUND(AVG(sl.breach_minutes),2) AS avg_breach_minutes
FROM ticket_sla_log sl
JOIN ticket t ON sl.ticket_id = t.ticket_id
JOIN company c ON t.company_id = c.company_id
LEFT JOIN app_user au ON t.assigned_agent_id = au.user_id
GROUP BY DATE_TRUNC('month', t.opened_at)::DATE, c.company_name, c.support_plan,
au.support_team, t.priority, sl.sla_type, sl.sla_status;

```

```

CREATE OR REPLACE VIEW vw_daily_kpi_summary AS
SELECT
    k.kpi_date,
    c.company_name,
    c.industry,
    c.region,
    c.support_plan,
    k.support_team,
    k.total_tickets,
    k.resolved_tickets,
    k.closed_tickets,
    k.backlog_tickets,
    k.sla_met_tickets,
    k.sla_breached_tickets,
    ROUND(100.0 * k.sla_met_tickets / NULLIF(k.sla_met_tickets +
k.sla_breached_tickets,0),2) AS sla_compliance_pct,
    k.avg_resolution_minutes,
    k.avg_first_response_minutes,
    k.csat_avg
FROM daily_itsm_kpi k
JOIN company c ON k.company_id = c.company_id;

```

```

CREATE OR REPLACE VIEW vw_management_itsm_dashboard AS
SELECT
    m.summary_month,
    c.company_name,
    m.region,
    m.support_plan,
    m.total_tickets,
    m.incident_count,
    m.service_request_count,
    m.high_priority_count,
    m.resolved_count,
    m.closed_count,
    m.backlog_count,
    m.sla_met_count,
    m.sla_breached_count,
    m.sla_compliance_pct,
    m.avg_resolution_hours,
    m.avg_csat,
    m.escalation_count,
    CASE
        WHEN m.sla_compliance_pct >= 95 THEN 'Excellent'
        WHEN m.sla_compliance_pct >= 85 THEN 'Good'
        WHEN m.sla_compliance_pct >= 75 THEN 'Needs Attention'
        ELSE 'Critical'
    END AS service_health_status
FROM management_itsm_summary m
JOIN company c ON m.company_id = c.company_id;

```

```

CREATE OR REPLACE VIEW vw_activity_log_all_months AS
SELECT '2021-April' AS activity_month, * FROM ticket_activity_log_2021_april
UNION ALL

```

```

SELECT '2021-May', * FROM ticket_activity_log_2021_may
UNION ALL
SELECT '2021-June', * FROM ticket_activity_log_2021_june
UNION ALL
SELECT '2021-July', * FROM ticket_activity_log_2021_july;

-- =====
-- 12. VALIDATION QUERIES
-- =====

SELECT 'company' AS table_name, COUNT(*) AS total_records FROM company
UNION ALL SELECT 'app_user', COUNT(*) FROM app_user
UNION ALL SELECT 'ticket', COUNT(*) FROM ticket
UNION ALL SELECT 'ticket_sla_log', COUNT(*) FROM ticket_sla_log
UNION ALL SELECT 'daily_itsm_kpi', COUNT(*) FROM daily_itsm_kpi
UNION ALL SELECT 'management_itsm_summary', COUNT(*) FROM management_itsm_summary
UNION ALL SELECT 'ticket_activity_log_2021_april', COUNT(*) FROM
ticket_activity_log_2021_april
UNION ALL SELECT 'ticket_activity_log_2021_may', COUNT(*) FROM ticket_activity_log_2021_may
UNION ALL SELECT 'ticket_activity_log_2021_june', COUNT(*) FROM
ticket_activity_log_2021_june
UNION ALL SELECT 'ticket_activity_log_2021_july', COUNT(*) FROM
ticket_activity_log_2021_july;

-- Suggested validation checks:
-- SELECT ticket_status, COUNT(*), COUNT(resolved_at), COUNT(closed_at) FROM ticket GROUP BY
ticket_status;
-- SELECT sla_status, COUNT(*) FROM ticket_sla_log GROUP BY sla_status;
-- SELECT * FROM vw_management_itsm_dashboard ORDER BY summary_month, total_tickets DESC
LIMIT 20;
-- =====
-- END OF SCRIPT
-- =====

```